

INT303 Lecture Summary

Big Data Analytics

Contents

1 Lecture 1: 大数据分析导论与核心流程 (Introduction & The Data Science Process)	9
1.1 模块一: 科学范式与大数据的本质	9
1.1.1 1. 科学研究范式的演变 (The Evolution of Science)	9
1.1.2 2. 数据分析师的角色 (The Data Analyst)	9
1.1.3 3. 核心隐喻 (Key Metaphor)	9
1.1.4 4. 数据的特征	10
1.2 模块二: 数据挖掘 (Data Mining) 的定义与核心	10
1.2.1 1. 什么是数据挖掘?	10
1.2.2 2. 学科交叉性 (Confluence of Disciplines)	10
1.2.3 3. 数据创造价值的实战领域 (Data Creates Value)	10
1.3 模块三: 数据科学全流程 (The Data Science Process) — 重点案例 Hubway	11
1.3.1 1. 核心循环 (The Cycle)	11
1.3.2 2. 假设提出的五个维度 (The 5 Ws)	11
1.3.3 3. 验证手段	11
1.4 模块四: 核心算法逻辑与社交网络分析	11
1.4.1 1. 利用相似性 (Exploiting Similarities)	11
1.4.2 2. 预测任务 (Prediction Tasks)	12
1.4.3 3. 社交图谱与图算法 (Social Graph)	12
1.5 模块五: 伦理与新时代挑战	12
1.6 讲师的考前叮嘱 (Exam Strategy)	13
2 Lecture 2: Data Foundations & Representation (数据基础与表示)	14
2.1 第一部分: 核心定义与流程 (The Basics)	14
2.1.1 1. 数据挖掘管道 (The Data Mining Pipeline)	14

2.1.2	2. 数据的基本构成 (Data Objects & Attributes)	14
2.1.3	3. 数据的宏观分类 (Relational Data)	14
2.2	第二部分: 属性的分类体系 (Taxonomy of Attributes)	14
2.2.1	1. 类别型 (Categorical)	14
2.2.2	2. 数值型 (Numeric)	15
2.3	第三部分: 数据转换与离散化 (Transformation & Binning)	15
2.3.1	1. 类别转数值 (Categorical $\rightarrow$ Numeric)	15
2.3.2	2. 数值转类别 (Numeric $\rightarrow$ Categorical) — 分箱 (Binning)	15
2.4	第四部分: 复杂数据结构与向量化 (Data Structures)	16
2.4.1	1. 集合数据 (Set Data)	16
2.4.2	2. 依赖型数据 (Dependent Data)	16
2.5	第五部分: 图数据 (Graph Data)	16
2.5.1	1. 定义	17
2.5.2	2. 三种表示形式 (Representations) — 优缺点对比必背	17
2.6	老师的最后叮嘱 (Final Exam Tips)	17
3	Lecture 3: The Data Mining Pipeline — 终极考前知识点清单	18
3.1	核心板块概览 (Module Overview)	18
3.2	第一部分: 核心计算与公式 (Formulas & Calculations)	18
3.2.1	A. 文本挖掘核心: TF-IDF (Text Mining)	18
3.2.2	B. 数据归一化: 列处理 (Column Normalization)	19
3.2.3	C. 数据归一化: 行处理 (Row Normalization)	19
3.2.4	D. 采样概率计算 (Sampling Probability)	20
3.3	第二部分: 核心概念与辨析 (Concepts & Logic)	20
3.3.1	A. 采样策略 (Sampling Strategies)	20
3.3.2	B. 数据清洗逻辑 (Data Cleaning)	21
3.3.3	C. 特征提取进阶 (Word Embeddings)	21
3.3.4	D. 降维 (Dimensionality Reduction)	21
3.4	第三部分: 统计陷阱与批判性思维 (Critical Thinking & Pitfalls)	21
3.4.1	A. 邦费罗尼原理 (Bonferroni's Principle)	22

3.4.2	B. 莱茵悖论 (Rhine Paradox)	22
3.4.3	C. 结果验证 (Sanity Check)	22
3.5	第四部分: 考试实战建议 (Exam Tips)	22
4	Lecture 4: 数据可视化核心逻辑与考点 (Data Visualization)	23
4.1	模块一: 可视化的核心理论与原则 (Theory & Principles)	23
4.1.1	1. 为什么要可视化 (Motivation) — Anscombe's Quartet (安斯库姆四重奏)	23
4.1.2	2. Edward Tufte 的三大黄金原则 (Principles)	23
4.2	模块二: 图表选择决策树 (Chart Selection Taxonomy)	24
4.2.1	1. Distribution (分布)	24
4.2.2	2. Relationship (关系)	24
4.2.3	3. Comparison (比较)	24
4.2.4	4. Composition (构成)	24
4.3	模块三: 统计图表深度解析 (Statistical Deep Dive)	25
4.3.1	1. Boxplots (箱线图) 的解剖学	25
4.3.2	2. 利用箱线图判断偏度 (Skewness)	25
4.4	模块四: 高维数据可视化策略 (High-Dimensional Strategy)	25
4.5	特别警示: 关于饼图 (The Pie Chart Trap)	26
4.6	考试答题套路总结 (Exam Tips)	26
5	Lecture 5: Web 数据获取与抓取技术 (Web Data Acquisition & Scraping Techniques)	27
5.1	模块一: 数据科学核心概念体系 (Theory Framework)	27
5.1.1	1. 数据科学韦恩图 (Data Science Venn Diagram)	27
5.1.2	2. 机器 vs 人类 (Machine vs Human Spectrum)	27
5.2	模块二: Web 基础设施与 HTTP 协议 (Infrastructure)	27
5.2.1	1. Web 服务器 (Web Server) 的定义	27
5.2.2	2. URL 结构拆解 (必考公式)	28
5.2.3	3. HTTP 状态码 (Status Codes) (死记硬背)	28
5.3	模块三: 爬虫原理与伦理 (Scraping Principles & Ethics)	28

5.3.1	1. 爬虫 (Scraper) vs 浏览器 (Browser)	28
5.3.2	2. 爬虫的三大挑战 (Challenges)	28
5.3.3	3. 法律与网络礼仪 (Legal & Netiquette) (红线)	29
5.4	模块四: Python 爬虫实战代码逻辑 (Implementation)	29
5.4.1	1. 核心库 (Libraries)	29
5.4.2	2. 查找元素的方法 (Find Methods) (核心算法逻辑)	29
5.4.3	3. 数据提取与清洗 (Extraction & Cleaning)	30
5.4.4	4. DOM 树导航 (Tree Navigation)	30
5.5	考试通关微操 (Exam Tips)	30
6	Lecture 6: 从单机瓶颈到分布式“存算一体”架构 (Distributed Systems)	31
6.1	第一板块: 硬件基础与核心挑战 (Infrastructure & Challenges)	31
6.1.1	1. 单机架构的局限 (Bottlenecks)	31
6.1.2	2. 集群架构参数 (Cluster Hierarchy)	31
6.1.3	3. 规模与故障 (Scale & Failure)	31
6.2	第二板块: 分布式存储系统 (Distributed File System — DFS)	31
6.2.1	1. 核心设计理念 (The “Big Idea”)	32
6.2.2	2. GFS / HDFS 架构组件	32
6.2.3	3. 典型使用模式 (Usage Pattern)	32
6.3	第三板块: MapReduce 编程模型原理 (The Model)	32
6.3.1	1. 三阶段数据流 (Data Flow)	32
6.3.2	2. 必考判断题 (True/False Traps)	33
6.4	第四板块: MapReduce 算法实战 (Algorithms)	33
6.4.1	1. Word Count (词频统计)	33
6.4.2	2. List/Join (关联/统计)	34
6.5	第五板块: 进化与对比 (Hadoop vs. Spark)	34
6.5.1	1. Spark 的定位	34
6.5.2	2. 终极对比 (The “Killer” Difference)	34
6.6	考前最后自检 (Self-Check)	35

7	Lecture 7: 高维数据灾难与 PCA 降维打击 (Dimensionality Reduction)	36
7.1	第一板块: 高维性与交互项 (The Problem Setting)	36
7.1.1	1. 维数灾难 (Curse of Dimensionality)	36
7.1.2	2. 交互项的组合爆炸	36
7.2	第二板块: PCA 的核心数学原理 (The Theory)	36
7.2.1	1. PCA 的终极目标	36
7.2.2	2. 主成分 (Principal Components, PCs) 的三大性质 (必背)	37
7.2.3	3. 几何解释	37
7.3	第三板块: PCA 的算法实现与计算 (The Algorithm)	37
7.3.1	1. 标准化 (Standardization) — 至关重要!	37
7.3.2	2. 线性代数实现步骤 (Step-by-Step)	37
7.4	第四板块: 主成分回归 (PCR) 与模型评估	38
7.4.1	1. PCR 的流程	38
7.4.2	2. 如何选择 M ? (超参数调优)	38
7.4.3	3. 解释方差比 (Explained Variance Ratio)	38
7.5	第五板块: 考试陷阱 与优缺点 (The “Gotchas”)	38
7.5.1	1. PCA/PCR 的缺点 (Cons) — 高频考点	38
7.5.2	2. PCA/PCR 的优点 (Pros)	39
7.6	老师的最后叮嘱 (Exam Tips)	39
8	Lecture 8: Clustering (聚类分析) — 终极考前复习清单	40
8.1	第一部分: 核心概念与“高维灾难” (Core Concepts & High-Dim Issues)	40
8.1.1	1. 聚类的定义 (Definition)	40
8.1.2	2. 高维空间的挑战 (Curse of Dimensionality)	40
8.2	第二部分: 数据表示与距离度量 (Representation & Distance)	40
8.2.1	三大对应关系 (必须要背)	40
8.3	第三部分: 层次聚类 (Hierarchical Clustering)	40
8.3.1	1. 两种策略	40
8.3.2	2. 核心步骤 (Key Operation)	41
8.3.3	3. 簇的表示与距离计算 (关键区分)	41

8.3.4	4. 停止条件 (Termination Condition)	41
8.4	第四部分: K-Means 算法 (K-Means Clustering)	41
8.4.1	1. 算法流程 (The Lloyd's Algorithm)	41
8.4.2	2. 如何选择 K 值 (手肘法 — The Elbow Method)	42
8.4.3	3. 初始化策略 (Picking Initial Points)	42
8.5	第五部分: 进阶算法 — 应对大数据与复杂形状	42
8.5.1	1. BFR 算法 (针对海量数据)	42
8.5.2	2. CURE 算法 (针对不规则形状)	42
8.6	第六部分: 算法复杂度对比 (Complexity Checklist)	43
8.7	老师的最后叮嘱 (Final Tips)	43
9	Lecture 9: 监督学习核心模型 (回归与分类) 终极复习清单	44
9.1	第一部分: 核心概念框架 (Conceptual Framework)	44
9.1.1	1. 监督学习 (Supervised Learning)	44
9.1.2	2. 数据预处理 (回归模型必做)	44
9.2	第二部分: 线性回归模型 (Linear Regression)	44
9.2.1	1. 模型定义	44
9.2.2	2. 核心计算公式 (背诵)	45
9.2.3	3. 解释性	45
9.3	第三部分: 分类模型 — 决策树 (Decision Trees)	45
9.3.1	1. 树的结构与逻辑	45
9.3.2	2. 节点分裂技术 (Splitting Techniques)	46
9.3.3	3. 纯度计算公式 (Calculation — 必须熟练手算)	46
9.4	第四部分: 模型评估体系 (Evaluation)	47
9.4.1	1. 混淆矩阵 (Confusion Matrix)	47
9.4.2	2. 核心指标公式	47
9.5	考试避坑指南 (Exam Tips)	47
10	Lecture 10: Analysis Models & Evaluation — 终极考试特化知识点清单	49
10.1	第一板块: 核心分类算法 (The “Big Three” Algorithms)	49

10.1.1	1. 最近邻分类器 (K-Nearest Neighbor, KNN)	49
10.1.2	2. 支持向量机 (SVM)	50
10.1.3	3. 逻辑回归 (Logistic Regression)	50
10.2	第二板块: 模型理论与泛化 (Theory & Generalization)	51
10.2.1	1. 偏差-方差权衡 (Bias-Variance Tradeoff)	51
10.2.2	2. 过拟合 (Overfitting) 与 欠拟合 (Underfitting)	51
10.3	第三板块: 核心评估指标 (Metric Calculation)	51
10.3.1	1. 混淆矩阵 (Confusion Matrix)	52
10.3.2	2. 必背计算公式	52
10.3.3	3. ROC 曲线与 AUC	52
10.4	第四板块: 复杂场景与实战策略 (Advanced & Practical)	53
10.4.1	1. 成本矩阵计算 (Classification Cost)	53
10.4.2	2. 数据切分方法 (Validation Methods)	53
10.4.3	3. 类别不平衡处理 (Class Imbalance)	53
10.5	老师的最后叮嘱 (Final Exam Tips)	54
11	Lecture 11: Word Embeddings & Skip-gram Model — 考前终极复习	55
11.1	一、模型架构与核心参数 (Architecture & Parameters)	55
11.1.1	1. 基本定义	55
11.1.2	2. 两大核心权重矩阵 (必须死记维度)	55
11.1.3	3. 最终词向量的选择	55
11.2	二、Skip-gram 算法逻辑流程 (Algorithm Logic)	56
11.2.1	1. 任务目标	56
11.2.2	2. 前向传播五步走 (The 5 Steps)	56
11.3	三、核心数学公式 (Key Formulas)	56
11.3.1	1. Softmax 函数 (归一化概率)	56
11.3.2	2. 目标函数 (Objective Function)	56
11.4	四、词向量的特性与应用 (Properties & Applications)	57
11.4.1	1. 语义相似度 (Similarity)	57
11.4.2	2. 线性类比关系 (Linear Analogies) [必考]	57

11.4.3 3. 语法与语义捕捉	57
11.5 老师的考前叮嘱 (Final Check)	57
12 Lecture 12: 推荐系统核心理论与算法逻辑 — 终极复习	59
12.1 第一部分: 核心商业概念与长尾理论 (The Long Tail)	59
12.1.1 1. 从稀缺到富饶 (Scarcity to Abundance)	59
12.1.2 2. 长尾理论 (The Long Tail)	59
12.2 第二部分: 形式化模型与数据获取 (Formal Model)	59
12.2.1 1. 效用矩阵 (Utility Matrix)	59
12.2.2 2. 数据获取方式	59
12.3 第三部分: 算法一 — 基于内容的推荐 (Content-Based)	59
12.3.1 1. 物品画像 (Item Profile) 与 TF-IDF (必考计算)	60
12.3.2 2. 用户画像 (User Profile)	60
12.3.3 3. 预测方法	60
12.4 第四部分: 算法二 — 协同过滤 (Collaborative Filtering)	60
12.4.1 1. 两种模式	60
12.4.2 2. 三大相似度计算公式 (Sim Metrics) — 死记硬背	61
12.5 第五部分: 最终评分预测 (Rating Prediction)	61
12.5.1 1. 核心公式 (加权平均)	61
12.5.2 2. Item-Item CF 计算流程 (考试高频题)	62
12.6 第六部分: 评估体系 (Evaluation)	62
12.6.1 1. RMSE (均方根误差)	62
12.6.2 2. Top-N 推荐评估 (分类问题)	62
12.7 第七部分: 算法优缺点对比 (Pros & Cons) — 简答题必背	62
12.8 考前最后提醒 (Final Exam Tips)	62

1 Lecture 1: 大数据分析导论与核心流程 (Introduction & The Data Science Process)

这是整门课的基石。考试中，这部分通常考察概念辨析 (MCQ)、流程理解 (简答) 以及基础算法逻辑 (如 PageRank, 推荐系统原理)。

1.1 模块一：科学范式与大数据的本质

(考点：理解为什么我们需要大数据分析，以及它的历史定位)

1.1.1 1. 科学研究范式的演变 (The Evolution of Science)

- **第一范式 (Empirical):** 经验描述。几千年前，基于观察（如数星星、节气），描述现象。
- **第二范式 (Theoretical):** 理论推导。几百年前，使用数学方程描述普遍规律 (e.g., $F = ma$, $v = v_0 + at$)。
- **第三范式 (Computational):** 计算模拟。20世纪，工程与计算机介入，模拟复杂系统（如汽车、电网）。
- **第四范式 (Data-Intensive):** 数据密集型科学。
 - **背景:** 20世纪后遇到的问题（如：吸烟致癌吗？肥料增产吗？）存在复杂因果性，无法用简单物理公式推导。
 - **解决方案:** 必须依赖收集和分析海量的观察性数据 (Observational data) 或实验数据 (Experimental data)。

1.1.2 2. 数据分析师的角色 (The Data Analyst)

公式: $\text{Data Analyst} = \text{Statisticians} + \text{Computer Engineering}$ 。

需要统计学基础（分析）加上计算机工程能力（处理海量数据）。

1.1.3 3. 核心隐喻 (Key Metaphor)

“Data is the new oil” (Clive Humby):

- **原始状态:** 数据本身像原油，未提炼时价值有限。
- **提炼过程:** 必须经过 Broken down (分解) 和 Analyzed (分析)。
- **产出:** 只有处理后才能驱动决策和盈利 (Value)。

1.1.4 4. 数据的特征

- **海量与普遍性**: “Everybody collects data” (从DNA到点击流)。
- **复杂性 (Complex)**: 多种类型 (Text, Image, Graph, Time series)。
- **互联性 (Interconnected)**: 不同源的数据 (如位置、社交关系、搜索记录) 交织在一起。

1.2 模块二: 数据挖掘 (Data Mining) 的定义与核心

(考点: 定义的准确性, 学科交叉图谱)

1.2.1 1. 什么是数据挖掘?

- **简易定义**: Input (Data) -> Data Mining (Processing) -> Output (Value)。
- **权威定义 (Charu Aggarwal) [背诵关键词]**:
 - Collecting (收集)
 - Processing (处理)
 - Analyzing (分析)
 - Gaining useful insights (获取有用的洞察)
- **本质**: 即使数据是免费且无处不在的, 核心能力在于如何提取价值。

1.2.2 2. 学科交叉性 (Confluence of Disciplines)

数据挖掘不是单一学科, 而是多个领域的交汇。

考试必记三大支柱:

- **Statistics (统计学)**: 提供数学基础和推断逻辑。
- **Machine Learning (机器学习)**: 提供自动建模和学习机制。
- **Algorithms (算法)**: 提供处理大规模数据的计算效率。

(辅助支撑: 数据库技术, 可视化, 模式识别)

1.2.3 3. 数据创造价值的实战领域 (Data Creates Value)

- **NLP**: 机器翻译、搜索自动补全 (Google)。
- **医疗**: 精准医疗 (基于基因型 Genotype)、药物副作用监测 (通过搜索查询关联)。
- **智慧城市/交通**: 自动驾驶 (Car is the next computer)、城市计算。

- **体育**: 德国队世界杯夺冠 (视频分析 - 减少控球时间)、哈登防守图 (可视化分析效率)。
- **危机管理**: 预测灾害路径 (粒子滤波等算法)。

1.3 模块三: 数据科学全流程 (The Data Science Process) — 重点案例 Hubway

(考点: 给定一个场景, 问你如何按照流程进行分析, 或者具体的分析维度)

1.3.1 1. 核心循环 (The Cycle)

Data Exploration (数据探索) ↔ Question Refinement (问题精炼)。

这是一个迭代的过程, 不是线性的。先看数据, 提出假设, 验证, 再修正问题。

1.3.2 2. 假设提出的五个维度 (The 5 Ws)

- **Who (用户画像)**: 性别、年龄、订阅类型 (Subscriber vs One-time)。
- **Where (空间/地理)**: 借出地点、商业区 vs 居民区、是否在景点旁。
 - 难点: 数据可能需要在不同表之间 Merged (合并)。
- **When (时间/时序)**: 周末 vs 工作日、高峰期、季节。
 - 关键考点: Feature Engineering (特征工程)。很多特征 (如“是否是周末”) 在原始数据中不存在, 必须通过算法从原始日期中提取 (Engineered) 出来。
- **Why (动机)**: 通勤 (Commute) vs 娱乐 (Recreation) vs 旅游 (Tourist)。
- **How (多变量关联 Modeling)**:
 - 结合多个变量: 天气如何影响借车? 人口统计学如何影响骑行时长?

1.3.3 3. 验证手段

Visualization (可视化): 如六边形分箱图 (Hexbin Plot) 展示“骑行距离 vs 时长”的密度分布, 验证假设。

1.4 模块四: 核心算法逻辑与社交网络分析

(考点: 这是技术含量最高的部分, 容易出计算逻辑题或原理题)

1.4.1 1. 利用相似性 (Exploiting Similarities)

聚类 (Clustering):

- **逻辑**: 将具有相似行为模式的用户分到一组。

- **案例:** 体育迷 (只看ESPN) vs 财经迷 (只看WSJ)。

推荐系统 (Recommendation Systems):

- **核心假设:** Similar users like items similarly (相似的用户对物品有相似的喜好)。
- **操作:** 填补矩阵中的缺失值 (Missing Entry)。如果 A 和 B 很像, 且 A 喜欢 X, 那么 B 很可能也喜欢 X。

1.4.2 2. 预测任务 (Prediction Tasks)

考试中必须能区分以下三种预测类型:

- **Regression (回归):** 预测一个实数值 (Real Value)。
 - Example: 预测明天的点击次数 (e.g., 500次)。
- **Binary Classification (二分类):** 预测 Yes/No。
 - Example: 用户是否会点击这个广告?
- **Multiclass Classification (多分类):** 预测多个类别中的一个。
 - Example: 这个帖子的主题是体育、政治还是娱乐?

1.4.3 3. 社交图谱与图算法 (Social Graph)

图的构成: 节点 (Node/User) + 边 (Edge/Connection)。

三元闭包原则 (Triadic Closure Principle):

- **定义:** 链接的建立倾向于闭合三角形。
- **应用:** 好友推荐 (Friendship Suggestion)。如果 Bob 和 Charlie 都认识 Alice, 系统会推荐 Bob 和 Charlie 认识。

PageRank 算法 (核心逻辑):

- **问题:** 谁是图中最重要的节点?
- **定义 (递归):** 一个节点的重要性, 取决于它是否被其他“重要”的节点所指向。
- **注意:** 不仅仅是看粉丝数量, 更要看粉丝的质量 (权重)。

1.5 模块五: 伦理与新时代挑战

(考点: 简答题或论述题方向)

新时代特征: AI = ML = Data Mining (边界模糊); 算法必须 Scale (规模化)。

The Dark Side (阴暗面):

- **Fairness:** 算法决策是否公平?
- **Filter Bubbles / Echo Chambers:** 信息茧房, 只推荐你想看的内容, 加剧极化。
- **Surveillance Capitalism (监控资本主义):** 利用隐私数据盈利。
- **Misinformation:** 假新闻传播。

1.6 讲师的考前叮嘱 (Exam Strategy)

- 区分 “**Observational**” vs “**Experimental**”: 虽然 Lecture 1 没展开, 但要知道 20 世纪科学很多依赖这两种数据。
- **特征工程 (Feature Engineering):** 这是一个高频术语。记得在 Hubway 案例中, “Is Weekend?” 不是原始数据, 是被 Engineered 出来的。
- **分类 vs 回归:** 看到 “Predict number of...” 选回归, 看到 “Predict Topic/Category” 选分类。
- **PageRank:** 不要只记“被指向”, 要记“被重要节点指向”。

2 Lecture 2: Data Foundations & Representation (数据基础与表示)

2.1 第一部分：核心定义与流程 (The Basics)

这部分通常考察选择题或简答题的第一问，属于送分题，必须拿满。

2.1.1 1. 数据挖掘管道 (The Data Mining Pipeline)

必须记住顺序，考试可能让你排序或填空：

1. Collection (数据收集)
2. Preprocessing (数据预处理) → 包含 Sampling, Feature extraction, Normalization
3. Mining (挖掘)
4. Post-processing (后处理)

2.1.2 2. 数据的基本构成 (Data Objects & Attributes)

公式符号：

- n (Size) = 对象 (Rows/Objects) 的数量。
- d (Dimensionality) = 属性 (Columns/Dimensions) 的数量。

同义词辨析 (考试常考词汇互换)：

- **Object:** Record, Point, Vector, Case, Entity, Instance, Sample.
- **Attribute:** Variable, Field, Characteristic, Feature, Dimension.

2.1.3 3. 数据的宏观分类 (Relational Data)

- **Dense (稠密):** 几乎没有 Null 值 (e.g., 数据库表, 人口普查)。
- **Sparse (稀疏):** 绝大多数值为 0 或 Null (e.g., 文本挖掘的词频矩阵, 购物篮分析)。→ 后续高维处理的重点。

2.2 第二部分：属性的分类体系 (Taxonomy of Attributes)

这是本章最容易混淆的考点，尤其是“看起来像数字但不是数字”的情况。

2.2.1 1. 类别型 (Categorical)

Nominal (名义型)：

- 定义: 仅仅是标签, 没有顺序, 不可比较大小。
- 例子: 颜色 (红/蓝), 职业, Zip Codes (邮政编码)。
- 考试陷阱: 邮编虽然是数字, 但求“平均邮编”没有意义, 所以它是 Nominal。

Ordinal (序数型):

- 定义: 有顺序, 但间距不固定 (不可做加减法)。
- 例子: 排名 (Rankings), 满意度 (Good, Fair, Bad), 衣服尺码 (S, M, L)。

2.2.2 2. 数值型 (Numeric)

- 定义: 可进行数学运算, 有几何意义 (距离、相似度)。
- Discrete (离散型): 计数值 (Counts), 通常为整数。
- Continuous (连续型): 测量值 (Temperature, Length), 通常为浮点数。
- 特殊情况: Binary/Boolean (0/1) $\rightarrow$ 既可视为 Categorical (True/False), 也可视为 Numeric (用于计算)。

2.3 第三部分: 数据转换与离散化 (Transformation & Binning)

这部分涉及具体的操作逻辑, 可能会考“为什么要这么做”或者“选哪种方法更好”。

2.3.1 1. 类别转数值 (Categorical $\rightarrow$ Numeric)

- 方法: One-Hot Encoding (独热编码)。
- 操作: 为每个可能的类别值创建一个新的 Binary 属性。
- 目的: 转换后可以计算距离, 可以使用线性代数工具。

2.3.2 2. 数值转类别 (Numeric $\rightarrow$ Categorical) — 分箱 (Binning)

目的: 降低数据粒度, 处理非线性关系。

三大分箱策略 (必考对比):

- Equi-width (等宽):
 - 定义/逻辑: 每个箱的数值跨度 (interval size) 相同。
 - 缺点: 受异常值影响大, 可能导致某些箱极度稀疏 (Sparse) 或为空。
- Equi-depth/size (等深):
 - 定义/逻辑: 每个箱包含的样本数量 (number of elements) 相同。
 - 特点: 基于分位数 (Quantiles)。

– **缺点:** 某些箱的数值范围可能极窄 (如果数据密集)。

- **Equi-log (等对数):**

- **定义/逻辑:** $\log(\text{end}) - \log(\text{start}) = C$

- **适用场景:** 偏态分布 (Skewed distributions), 如收入分布 (大部分人穷, 极少数人富)。

2.4 第四部分: 复杂数据结构与向量化 (Data Structures)

理解如何将非表格数据转化为矩阵形式。

2.4.1 1. 集合数据 (Set Data)

典型: 购物篮 (Transactions), 文档 (Documents)。

向量化表示 (Vector Representation):

- **维度:** 所有可能项 (Items) 或 所有可能词 (Words) 的总数。
- **值:**
 - 购物篮: Binary (0/1) → 买没买。
 - 文档: Counts (计数) → 词频。
- **特性:** 极度稀疏 (Sparsity) → 大部分元素是 0。

2.4.2 2. 依赖型数据 (Dependent Data)

定义: 对象之间不再独立, 存在显式依赖。

分类对比:

- **Time Series (时间序列):** 数值型 随时间变化 (e.g., 股票价格)。
- **Sequence Data (序列数据):** 类别型 随时间/顺序变化 (e.g., 基因序列 ACGT, 服务器日志)。
- **Spatial (空间):** 固定位置的数据 (e.g., 地图上的温度)。
- **Spatiotemporal (时空):**
 - 固定位置随时间变化 (气象站)。
 - 位置随时间移动 (Trajectories/轨迹)。

2.5 第五部分: 图数据 (Graph Data)

这一块是这一章最硬核的知识点, 涉及数据结构, 必考。

2.5.1 1. 定义

- **构成:** $G = (V, E)$, 实体 (Nodes/Entities) + 关系 (Edges/Links)。
- **类型:**
 - Undirected (无向): 关系对等 (e.g., Facebook 好友)。
 - Directed (有向): 关系有方向 (e.g., URL 链接, Twitter 关注)。

2.5.2 2. 三种表示形式 (Representations) — 优缺点对比必背

Adjacency Matrix (邻接矩阵):

- **数据结构描述:** $n \times n$ 矩阵, $A_{ij} = 1$ 表示有边。
- **优点 (Pros):** 概念清晰, 便于线性代数计算。
- **缺点 (Cons):** 空间浪费 (Wasteful), 极度稀疏。

Adjacency List (邻接表):

- **数据结构描述:** 每个节点挂一个 List, 存邻居 ID。
- **优点 (Pros):** 节省空间。
- **缺点 (Cons):** 维护困难 (不易增删改)。

List of Pairs (边列表):

- **数据结构描述:** 直接存 (i, j) 对的列表。
- **优点 (Pros):** 最简单, 最高效 (大数据并行处理首选)。
- **缺点 (Cons):** 查找特定边的速度不如矩阵快。

2.6 老师的最后叮嘱 (Final Exam Tips)

- **万物皆矩阵:** 这一章的核心思想是, 无论是什么乱七八糟的数据 (文本、图、集合), 我们最终都要想办法把它变成 Matrix (Rows=Objects, Columns=Attributes), 以便计算机处理。
- **Sparsity (稀疏性):** 这是一个贯穿全课程的关键词。在 Shopping Basket 和 Document Data 中, 稀疏是常态, 不是异常。
- **Zip Code:** 再强调一次, 看到 Zip Code 选 Categorical/Nominal, 不要犹豫。

3 Lecture 3: The Data Mining Pipeline — 终极考前知识点清单

3.1 核心板块概览 (Module Overview)

本章考试的核心在于预处理 (Preprocessing) 的计算与逻辑。

权重分布预测:

- **计算题 (High Probability):** TF-IDF 计算、归一化 (Min-Max, Z-score) 计算、采样概率计算。
- **概念题:** 采样偏差、清洗策略、行vs列归一化的区别、统计显著性陷阱。

3.2 第一部分: 核心计算与公式 (Formulas & Calculations)

这部分是**必须死记硬背**并在草稿纸上能推导的。

3.2.1 A. 文本挖掘核心: TF-IDF (Text Mining)

这是将非结构化文本转化为结构化数值矩阵的核心算法。

基本逻辑: 一个词对文档的重要性 = (该词在文档中的频率) $\times$ (该词在整个语料库中的稀缺度)。

公式拆解:

TF (Term Frequency): 单词 w 在文档 d 中出现的次数。

$$TF(w, d) = \text{count}(w \text{ in } d)$$

IDF (Inverse Document Frequency): 衡量词的独特性。

$$IDF(w) = \log \left(\frac{D}{D(w)} \right)$$

其中: D = 文档总数, $D(w)$ = 包含单词 w 的文档数。

特例:

- 若词在所有文档都出现 ($D(w) = D$), 则 $IDF = 0$ (如停用词)。
- 若词只在1个文档出现, IDF最大, 为 $\log(D)$ 。

TF-IDF 最终公式:

$$TF-IDF(w, d) = TF(w, d) \times IDF(w)$$

考点提示：考试可能会给出一个具体的短文档集合，让你计算某个特定单词的 TF-IDF 值。注意 log 的底数通常默认为 10 或 2，具体看题目要求，但逻辑不变。

3.2.2 B. 数据归一化：列处理 (Column Normalization)

用于消除不同特征 (Feature) 量纲/范围差异带来的影响 (例如：年龄 vs 收入)。

Min-Max Normalization (最小-最大归一化)

- 目的：将数据严格压缩到 $[0, 1]$ 区间。
- 公式：

$$NewValue = \frac{Value - Min}{Max - Min}$$

- 特征： $Max \rightarrow 1, Min \rightarrow 0$ 。

Z-Score Normalization (标准化)

- 目的：处理正态分布数据，将均值变为0，标准差变为1。
- 公式：

$$NewValue = \frac{Value - \mu}{\sigma}$$

- 特征：表示该数值距离均值有多少个标准差。 μ 是均值， σ 是标准差。

Log Transformation (对数变换)

- 场景：处理长尾分布 (Long-tail) 或 偏态分布 (Skewed) 数据 (如收入数据，贫富差距极大)。
- 作用：拉近极端值，使数据分布更接近正态分布，避免小数值被忽略。

3.2.3 C. 数据归一化：行处理 (Row Normalization)

用于消除个体记录 (Record/User) 自身习惯差异带来的影响 (例如：打分严苛的用户 vs 手松的用户)。

行中心化 (Centering)

- 公式： $Value_{new} = Value_{old} - RowMean$
- 作用：消除“平均偏差”。正值代表“高于该用户的平均水平”，负值代表“低于”。能够揭示不同打分习惯用户的一致偏好。

概率转化 (Probability Transformation)

Softmax 函数 (用于强调单一选择, 拉大差距):

$$P(i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Sigmoid/Logistic 函数 (映射到 $[0,1]$, 用于二分类或概率):

$$\phi(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid 进阶: $f(x) = \frac{1}{1 + e^{-c_1(x - c_2)}}$

- c_1 : 控制斜率 (Slope)。
- c_2 : 控制中心点 (0.5 point)。

3.2.4 D. 采样概率计算 (Sampling Probability)

场景: 从 N 个球中取 k 个。

- **放回采样 (With Replacement):** 每次抽取概率不变, 独立事件。 $P(A, B) = P(A) \times P(B)$ 。
- **不放回采样 (Without Replacement):** 分母会减小。 $P(A, B) = P(A) \times P(B|A)$ 。

例题逻辑: 100人中51女49男, 抽2人都是女性的概率?

- 放回: 0.51×0.51
- 不放回: $\frac{51}{100} \times \frac{50}{99}$

3.3 第二部分: 核心概念与辨析 (Concepts & Logic)

3.3.1 A. 采样策略 (Sampling Strategies)

为什么要采样? : 数据太大 (处理慢) 或 数据获取太贵。

核心原则: 样本必须具有代表性 (Representative), 否则会有偏差 (Bias)。

分层采样 (Stratified Sampling):

- **针对问题:** 数据不平衡 (Imbalanced Data)。例如欺诈检测 (只有0.1%是欺诈)。
- **操作:** 将数据分组, 从每组中分别采样。确保稀有类别 (欺诈) 也被抽到足够数量。

有偏采样 (Biased Sampling):

- **特定场景：**时间序列数据。
- **操作：**刻意引入 Recency Bias（近因偏差），赋予最近的数据更高的采样概率（ $p \propto e^{-t}$ ）。

3.3.2 B. 数据清洗逻辑 (Data Cleaning)

处理缺失值 (Missing Values) 策略：

- **删除：**最简单，但可能丢失信息。
- **填充均值/随机值：**简单，但可能破坏数据分布。
- **KNN (最近邻) / 簇均值：**最推荐。找最相似的记录（Feature最像的）的值填进去。
- **推断 (Inference)：**利用列与列之间的逻辑关系（如 $\text{Column C} = 2 * \text{Column A}$ ）。

处理离群点 (Outliers)：

- 不要盲目删除！有时候离群点（如欺诈交易、百万富翁）正是挖掘目标。
- 如果不重要（如输入错误），则删除或修正。

3.3.3 C. 特征提取进阶 (Word Embeddings)

Word2Vec:

- 区别于 TF-IDF（只看统计），Word2Vec 关注 上下文 (Context)。
- **核心假设：**出现在相似上下文中的词，其含义相似，对应的向量距离也近。

3.3.4 D. 降维 (Dimensionality Reduction)

- **矩阵分解 (Matrix Factorization)：**将大矩阵 D 分解为权重矩阵和基向量矩阵。
- **可视化降维：**t-SNE (非线性，SOTA方法)，用于将高维数据映射到 2D/3D 进行可视化。

3.4 第三部分：统计陷阱与批判性思维 (Critical Thinking & Pitfalls)

这部分常出现在简答题或判断题中，考察你对数据挖掘结果可靠性的理解。

3.4.1 A. 邦费罗尼原理 (Bonferroni's Principle)

核心含义：如果你在海量数据中寻找模式，且尝试的次数足够多，你一定能找到某种“模式”，但那可能只是随机性的产物（纯属巧合）。

考点：数据越大，越容易出现虚假关联。

3.4.2 B. 莱茵悖论 (Rhine Paradox)

案例：Joseph Rhine 发现 1/1000 的人能连续猜对10张牌，认为他们有超能力。重测后超能力消失。

错误原因：

- 幸存者偏差：在大样本中，必然有人靠运气全对 ($1/2^{10} \approx 1/1000$)。
- 均值回归：极端的运气不可持续，第二次测试必然回归到随机概率（50%）。

启示：必须定义正确的零假设 (Null Hypothesis)，验证结果是否显著优于随机猜测。

3.4.3 C. 结果验证 (Sanity Check)

- **Too good to be true**: 结果太完美（如100%准确率）→ 检查是否有数据泄露（如标签包含在特征中）。
- **Too bad to be true**: 结果比瞎猜还差 → 检查是否有数据对齐错误（如 ID 错位）。

3.5 第四部分：考试实战建议 (Exam Tips)

- 看到“Imbalanced Data”（不平衡数据）→ 马上想到 Stratified Sampling（分层采样）。
- 看到“Document Similarity”（文档相似度）→ 马上想到 TF-IDF 和 Row Normalization（消除长短文档差异）。
- 看到“User Rating”（用户打分）→ 马上想到 Row Centering（消除打分习惯差异）。
- 看到“Long Tail / Income”（长尾/收入）→ 马上想到 Log Transformation（对数变换）。
- 看到“Text to Vector”（文本转向量）→ 步骤是：Cleaning -> Tokenization -> Stopword Removal -> TF-IDF。

4 Lecture 4: 数据可视化核心逻辑与考点 (Data Visualization)

这份清单将本章内容重构为四大核心考点模块。请确保你对每一个加粗的概念都有清晰的记忆。

4.1 模块一：可视化的核心理论与原则 (Theory & Principles)

(考点权重：常考形式：选择题、简答题、判断图表好坏)

4.1.1 1. 为什么要可视化 (Motivation) — Anscombe's Quartet (安斯库姆四重奏)

核心现象：四组数据集拥有完全相同的简单统计摘要 (Sum, Mean, Std Dev 均相同)。

视觉差异：

- **Dataset I:** 线性关系。
- **Dataset II:** 曲线（非线性）关系。
- **Dataset III:** 线性，但有一个明显的离群点 (Outlier)。
- **Dataset IV:** X值几乎不变，仅受一个极端离群点影响。

考试结论：“Summary statistics clearly don't tell the story.” 单纯的统计数据会掩盖数据的分布结构和异常，必须结合可视化来发现 Patterns (模式) 和 Outliers (异常值)。

4.1.2 2. Edward Tufte 的三大黄金原则 (Principles)

这是判断图表优劣的唯一标准，背下来：

Maximize Data-to-Ink Ratio (最大化数据墨水比)：

- **含义：**Show the data。图表中每一滴墨水（像素）都应服务于数据展示。
- **操作：**去除不必要的背景颜色、网格线、边框。

Don't Lie With Scale (不要在比例尺上撒谎)：

- **核心概念：**Lie Factor (谎言因子)。
- **典型陷阱：**Y轴必须从 0 开始。如果 Y 轴被截断（例如从 5.0 开始），微小的差异会被视觉放大，造成误导。

Minimize Chart-Junk (最小化图表垃圾)：

- **含义：**Show data variation, not design variation。

- **典型反例：**3D Charts（尤其是 3D 柱状图）。3D 效果会扭曲视觉感知，分散注意力，属于“垃圾”。应始终优先选择 2D 平面图。

4.2 模块二：图表选择决策树 (Chart Selection Taxonomy)

(考点权重：常考形式：Scenario题 — 给定场景/数据类型，让你推荐图表)

你需要根据 “What would you like to show?” 将需求迅速归类为以下四种：

4.2.1 1. Distribution (分布)

目的：研究定量数值如何分布，识别 Central tendency (集中趋势)、Range (范围) 和 Outliers (异常值)。

核心图表：

- **Histogram (直方图)：**用于一维数据。
 - **关键考点 (Bin Sensitivity)：**直方图的形状对 Bins (箱子) 的数量非常敏感。Bins 太多或太少都会误导解读。
- **Boxplot (箱线图)：**用于比较多组数据的分布（详见模块三）。

4.2.2 2. Relationship (关系)

目的：发现变量间的 Correlations (相关性) 和 Clusters (聚类)。

核心图表：

- **Scatter Plot (散点图)：**双变量关系的标准解。
- **Bubble Chart (气泡图)：**三维及以上变量（通过 Size 编码第三维度）。

4.2.3 3. Comparison (比较)

目的：比较数值的 Magnitude (量级)，识别最高/最低值。

核心图表：

- **Bar Chart (柱状图)：**侧重于 Items (项目) 之间的比较。
- **Line Chart (折线图)：**侧重于体现 Continuity (连续性)，通常用于时间序列。

4.2.4 4. Composition (构成)

目的：部分与整体的关系 (Part-to-Whole)。

核心图表：

- **Static (静态):** Pie Chart (慎用), Stacked 100% Bar Chart。
- **Time-series (随时间):** Stacked Area Chart (堆叠面积图), Stacked Bar Chart。

4.3 模块三：统计图表深度解析 (Statistical Deep Dive)

(考点权重：常考形式：读图题 — 给你一个箱线图让你分析数据的偏度或计算范围)

4.3.1 1. Boxplots (箱线图) 的解剖学

你必须能精准读出箱线图的五个关键点：

- **Median (中位数/Q2):** 箱体中间的线。
- **Q1 (25th Percentile):** 箱体下底。
- **Q3 (75th Percentile):** 箱体上顶。
- **IQR (Interquartile Range):** 箱体高度 = $Q3 - Q1$ 。
- **Whiskers (须/边缘):** 通常定义为 $[Q1 - 1.5 \times IQR]$ 到 $[Q3 + 1.5 \times IQR]$ 的范围。
- **Outliers (异常值):** 超出 Whiskers 范围的独立点。

4.3.2 2. 利用箱线图判断偏度 (Skewness)

这是一个非常容易混淆的考点，请死记硬背以下规律：

- **Symmetric (对称/正态):** Median 线位于箱体正中央。
- **Left-Skewed (左偏/负偏):** 数据尾巴向左（低值方向）拖长。箱线图中，Median 线靠向 Q3 (Top/Right)，即靠上/靠右。
- **Right-Skewed (右偏/正偏):** 数据尾巴向右（高值方向）拖长。箱线图中，Median 线靠向 Q1 (Bottom/Left)，即靠下/靠左。

4.4 模块四：高维数据可视化策略 (High-Dimensional Strategy)

(考点权重：常考形式：简答题 — 如何在二维平面展示 4 维数据?)

当数据维度 ≥ 2 时，直接画 3D 散点图通常是无效的 (Impossible/Unhelpful)。

解决方案 (Mapping Variables to Visual Attributes):

- **Size (大小):** 用于编码 Quantitative (定量) 属性（例如 Bubble Chart）。
- **Color (颜色):** 用于编码 Categorical (分类) 属性（例如 Iris 数据集中用颜色区分花种类）。
- **Shape (形状):** 用于编码 Categorical (分类) 属性。

- **Small Multiples (小型多图)**: 将复杂数据拆解为多个低维度的图表并列展示。
 - 原理: Generate multiple plots of lower dimensionality.
 - 优势: 比单一的复杂图表更容易发现关系。

4.5 特别警示: 关于饼图 (The Pie Chart Trap)

考点: “Pie charts are often frowned upon.” (饼图常被嫌弃)

原因: 人类视觉对面积和角度的比较能力远弱于对长度 (Length) 的比较能力。

替代方案: 通常使用 Bar Chart (柱状图) 来替代饼图进行构成或比较分析。

4.6 考试答题套路总结 (Exam Tips)

若题目问“为什么这张图不好?”

- 检查是否违反 Tufte 原则 (是否有 Chart-junk? Y轴是否从0开始? 墨水比是否低?)。
- 检查是否用了 Pie Chart 或 3D Chart。

若题目问“我有 [变量A, 变量B, 变量C], 该用什么图?”

- 第一步: 判断变量类型 (定量 vs 分类)。
- 第二步: 判断目的 (分布/关系/比较/构成)。
- 第三步: 套用决策树 (例如: 3个变量 + 找关系 = Bubble Chart; 随时间变化 + 构成 = Stacked Area Chart)。

若题目给 Boxplot 问分布形态

- 看中位数线的位置。
- 线靠下 (Q1) = 右偏; 线靠上 (Q3) = 左偏。

5 Lecture 5: Web 数据获取与抓取技术 (Web Data Acquisition & Scraping Techniques)

5.1 模块一：数据科学核心概念体系 (Theory Framework)

这部分通常出现在选择题或概念填空题中，考察你对学科定位的理解。

5.1.1 1. 数据科学韦恩图 (Data Science Venn Diagram)

三大核心技能 (必须背诵):

- **Hacking Skills (黑客/编程技能)**: 对应数据工程。
- **Math & Statistics Knowledge (数学与统计知识)**: 对应数据分析。
- **Substantive Expertise (领域专业知识)**: 对应商业智能。

关键交叉区域 (**高频考点**):

- **Machine Learning (机器学习)**: Hacking Skills + Math & Statistics。
- **Traditional Research (传统研究)**: Math & Statistics + Substantive Expertise。
- **Danger Zone (危险区域)**: Hacking Skills + Substantive Expertise。

考试逻辑: 为什么叫危险区? 因为它缺乏数学/统计学的严谨性。你会写代码处理数据, 也懂业务, 但可能得出统计学上不成立的错误结论。

5.1.2 2. 机器 vs 人类 (Machine vs Human Spectrum)

- **机器侧 (Machine)**: 关注 Data Management (数据管理) 和 Data Mining (数据挖掘)。
- **人类侧 (Human)**: 关注 Cognition (认知) 和 Story Telling (叙事)。
- **连接点**: Visualization (可视化) 是连接机器数据与人类感知的桥梁。

5.2 模块二：Web 基础设施与 HTTP 协议 (Infrastructure)

这部分是理解爬虫原理的基础，考察形式多为“判断 URL 结构”或“解释状态码含义”。

5.2.1 1. Web 服务器 (Web Server) 的定义

- 是一个 Daemon (守护进程), 长期运行。
- 在 Pre-specified port (预指定端口) 监听。
- 使用 HTTP 协议 响应请求。

5.2.2 2. URL 结构拆解 (必考公式)

给定 URL: `http://localhost:8888/Documents/cs109/BLA.ipynb#something`

- **Protocol (协议):** `http`
- **Hostname (主机名):** `localhost` (表示本机)
- **Port (端口):** `8888`
- **Path (路径):** `/Documents/cs109/BLA.ipynb` (资源在服务器的位置)
- **Fragment (片段):** `#something` (页面内的锚点位置)

5.2.3 3. HTTP 状态码 (Status Codes) (死记硬背)

看到代码必须立刻反应出含义:

- **200 OK:** 成功。一切正常。
- **400 Bad Request:** 客户端请求语法错误, 服务器看不懂。
- **401 Unauthorized:** 未授权 (通常缺登录信息, 可能修复)。
- **403 Forbidden:** 禁止访问 (即便登录了也不行, 绝对禁止)。
 - 辨析: 考试常考 401 和 403 的区别。
- **404 Not Found:** 找不到资源 (死链)。
- **500 Internal Server Error:** 服务器内部崩溃 (不是你的错, 是服务器的错)。

5.3 模块三: 爬虫原理与伦理 (Scraping Principles & Ethics)

这部分考察你是否知道“什么能做, 什么不能做”。

5.3.1 1. 爬虫 (Scraper) vs 浏览器 (Browser)

- **浏览器:** 下载 Code + Images + Style -> Rendering (渲染成图形界面) -> 给人看。
- **爬虫:** 发送 Request -> 下载 Code -> 丢弃 Images/Style -> 提取纯 Data -> 给程序用。

5.3.2 2. 爬虫的三大挑战 (Challenges)

- **Which data? (数据选择难):** 难以判断相关性和可靠性。
- **Dynamic? (动态性):** 网页结构 (HTML Structure) 随时会变, 代码易失效。
- **Volatile? (易变性):** 数据本身的模式随时间改变。

5.3.3 3. 法律与网络礼仪 (Legal & Netiquette) (红线)

Robots.txt (机器人协议):

- **位置:** 网站根目录/robots.txt。
- **作用:** 规定了哪些页面允许爬, 哪些禁止。必须遵守。
- **User-Agent:** 必须在代码中表明身份, 不要伪装。
- **隐私原则:** 只抓取 Public sources (公开来源)。
- **性能原则:** 设置间隔 (Idle time), Do not overload servers (不要让服务器过载/DDoS)。

5.4 模块四: Python 爬虫实战代码逻辑 (Implementation)

这是最重要的部分, 涉及代码填空、纠错或逻辑解释。

5.4.1 1. 核心库 (Libraries)

- **Requests:** 用于发送 HTTP 请求, 获取 HTML 源码。
 - `requests.get(URL)`
- **BeautifulSoup (bs4):** 用于解析 HTML, 提取数据。
 - `BeautifulSoup(page.content, "html.parser")`

5.4.2 2. 查找元素的方法 (Find Methods) (核心算法逻辑)

`soup.find(id="...")`:

- **返回:** 单个对象 (Object)。
- **场景:** 查找唯一的元素 (ID 是唯一的)。

`soup.find_all("tag", class_="...")`:

- **返回:** 列表 (List/Iterable)。注意是复数!
- **场景:** 查找一类元素。
- **陷阱:** HTML 中的属性叫 class, 但在 Python 中必须写成 class_ (带下划线), 因为 class 是 Python 保留字。

按文本查找 (String Matching):

- **精确匹配:** `string="Python"`
- **模糊匹配 (Lambda):** `string=lambda text: "python" in text.lower()` (不区

分大小写)。

5.4.3 3. 数据提取与清洗 (Extraction & Cleaning)

假设我们已经找到了一个元素对象 `element`:

- 获取可见文字: `element.text`
- 去除空白: `element.text.strip()` (去除首尾的空格、换行符 `\n`)。
- 获取链接/属性: `element['href']` (像字典一样访问属性)。

5.4.4 4. DOM 树导航 (Tree Navigation)

- 向下: `.find()` / `.find_all()`
- 向上 (反向): `element.parent`
- 场景: 先找到了内部的“职位名称”, 想获取包裹它的“整个职位卡片”时, 使用 `.parent` 或 `.parent.parent`。

5.5 考试通关微操 (Exam Tips)

- 代码填空时: 如果看到变量名是复数 (如 `jobs`), 大概率要填 `find_all`; 如果是单数, 填 `find`。
- 属性陷阱: 只要考到按 Class 查找, 一定检查有没有写下划线 `class_`。
- 层级逻辑: 记住步骤是 Request (获取) -> Parse (解析) -> Find (定位) -> Extract (提取)。顺序不能乱。
- 状态码: `4xx` 是客户端 (你) 的问题, `5xx` 是服务器的问题。

6 Lecture 6: 从单机瓶颈到分布式“存算一体”架构 (Distributed Systems)

这份清单分为五大核心板块。请按此顺序进行最终记忆扫描。

6.1 第一板块：硬件基础与核心挑战 (Infrastructure & Challenges)

这一部分通常考选择题或填空题，考察对数据中心物理限制的理解。

6.1.1 1. 单机架构的局限 (Bottlenecks)

- **计算/内存密集型任务**：如机器学习 (Machine Learning) 和统计 (Statistics)，瓶颈在 CPU 和 Memory。
- **I/O 密集型任务**：如经典数据挖掘 (Classical Data Mining)，瓶颈在 Disk (磁盘读写速度)。

6.1.2 2. 集群架构参数 (Cluster Hierarchy)

构成：Node (节点) → Rack (机架, 16-64个节点) → Cluster (集群)。

带宽瓶颈 (必须背)：

- **机架内 (Intra-rack)**：约 1 Gbps (快)。
- **机架间 (Inter-rack)**：约 2-10 Gbps (共享带宽，这是主要瓶颈)。

考试推论：尽量减少跨机架的数据传输。

6.1.3 3. 规模与故障 (Scale & Failure)

常态化故障：在大数据中，故障是常态 (Norm)，不是异常。

故障率数据 (以 Google 为例)：

- 1台机器寿命 $\approx$ 3年 (1000天)。
- 100万台机器的集群 → 每天 约有 1000台 机器故障。

核心应对思想：软件必须具备容错性 (Fault Tolerance)。

6.2 第二板块：分布式存储系统 (Distributed File System — DFS)

这一部分考察如何“存”数据。重点在于架构组件和可靠性机制。

6.2.1 1. 核心理念 (The “Big Idea”)

- **计算向数据移动 (Bring computation to data):** 移动代码比移动 PB 级的数据便宜得多。
- **冗余存储 (Replication):** 用空间换可靠性。

6.2.2 2. GFS / HDFS 架构组件

Chunk Servers (数据节点):

- **职责:** 存储实际的数据块 (Chunks)。同时也是计算节点 (Compute Server)。
- **Chunk 特性:** 文件被切分为连续的块, 大小通常 16-64 MB (或更大)。
- **复制机制:** 每个 Chunk 默认复制 2-3份, 且存放在不同的机架上 (防止机架断电导致数据丢失)。

Master Node / Name Node (主节点):

- **职责:** 存储 Metadata (元数据)。即“哪个文件的哪个块在哪个服务器上”。
- **特点:** 通常存在内存中, 是单点, 也需要备份。

Client (客户端):

- **交互流程:** 先问 Master 拿地址 → 直接连接 Chunk Server 读数据 (数据流不经过 Master)。

6.2.3 3. 典型使用模式 (Usage Pattern)

- 文件巨大 (GB~TB级)。
- **读多写少:** 数据极少“原地更新” (Rarely updated in place), 主要是追加 (Append) 和读取。

6.3 第三板块: MapReduce 编程模型原理 (The Model)

这是全章最核心内容, 涉及原理理解和流程图。

6.3.1 1. 三阶段数据流 (Data Flow)

Map (映射):

- **输入:** k1: v1 (如文档的一行)。
- **输出:** list(k2: v2) (中间键值对, 如单词计数 word: 1)。
- **并行性:** 多个 Map Task 并行处理。

Group by Key (分组/Shuffle & Sort):

- 系统自动完成 (不可见): 框架负责将所有 Map 输出中 Key 相同的 Value 抓取到一起。
- 输出: $k2: \text{list}(v2)$ 。

Reduce (归约):

- 输入: $k2: \text{list}(v2)$ (某一个 Key 和它所有的值)。
- 输出: $k3: v3$ (最终聚合结果)。

6.3.2 2. 必考判断题 (True/False Traps)

- 误区1: Map/Reduce 的输入输出数量必须一致? → 错 (False)。
 - 解释: Map 可以过滤数据 (变少) 或拆分数据 (变多); Reduce 通常聚合数据 (变少)。
- 误区2: Map/Reduce 的输入输出类型必须一致? → 错 (False)。
 - 解释: 输入可以是 String, 输出可以是 Integer。
- 误区3: Reducer 接收的数据是按 Key 分组的? → 对 (True)。
 - 解释: 这是 MapReduce 的定义, 保证相同 Key 进同一个 Reducer。
- 误区4: 可以在 Map 还没结束时就开始 Reduce? → 错 (False)。
 - 解释: 必须等所有 Map 完成 (Barrier), 才能保证 Reducer 拿到该 Key 的全部数据。

6.4 第四板块: MapReduce 算法实战 (Algorithms)

考试可能让你写伪代码或描述逻辑。

6.4.1 1. Word Count (词频统计)

Map:

- 输入: $(\text{doc_id}, \text{text})$
- 逻辑: 遍历 text 中的单词 w 。
- 输出: $\text{emit}(w, 1)$

Reduce:

- 输入: $(\text{word}, [1, 1, 1, \dots])$

- **逻辑:** `sum(values)`
- **输出:** `emit(word, count)`

6.4.2 2. List/Join (关联/统计)

场景: 输入 (Person, Item), 统计每个 Item 有多少人买。

Map:

- **逻辑:** 提取 Item。
- **输出:** `emit(Item, 1)` (或者保留 Person 信息则 `emit(Item, Person)`)

Reduce:

- **输入:** (Item, [1, 1, ...])
- **逻辑:** `sum` 列表长度。
- **输出:** (Item, Total_Count)

6.5 第五板块: 进化与对比 (Hadoop vs. Spark)

这是“未来技术”考点, 重点在于对比优劣。

6.5.1 1. Spark 的定位

- **Extends MapReduce:** 继承了 MR 的思想, 但改进了实现。
- **RDD:** Resilient Distributed Dataset (弹性分布式数据集), Spark 的核心数据抽象。
- **多语言:** 原生支持 Java, Scala, Python。

6.5.2 2. 终极对比 (The “Killer” Difference)

Hadoop MapReduce:

- **Disk-based (基于磁盘):** Map 结果写磁盘 → Reduce 读磁盘 → 结果写磁盘。
- **缺点:** 迭代计算 (如机器学习) 非常慢, 因为一直在进行磁盘 I/O。

Spark:

- **Memory-based (基于内存):** 中间结果保存在 内存 (RAM) 中。
- **优势:** 速度极快 (10x - 100x faster)。

6.6 考前最后自检 (Self-Check)

- 能不能画出 MapReduce 中间那个 Group by Key 的作用? (把分散的 Key 聚在一起)
- 如果问你为什么 Google 要用普通 PC 服务器而不是超级计算机? (因为性价比高, 通过软件容错来解决硬件故障)
- Spark 为什么比 Hadoop 快? (关键词: In-Memory vs. Disk-based)
- HDFS 的 Chunk 存在哪里? Metadata 存在哪里? (Chunk Server vs. Master Node)

7 Lecture 7: 高维数据灾难与 PCA 降维打击 (Dimensionality Reduction)

7.1 第一板块: 高维性与交互项 (The Problem Setting)

这个板块通常考选择题或简答题, 考察你对“为什么要降维”的理解。

7.1.1 1. 维数灾难 (Curse of Dimensionality)

大数据的定义陷阱: 统计学上的“大数据”不仅指观测值 n 大, 更关注预测变量 p 大。

高维性 (High Dimensionality): 当 p 很大, 甚至 $p \approx n$ 或 $p > n$ 时。

高维带来的后果 (必背):

- 矩阵不可逆: OLS 无法计算 (Matrices may not be invertible)。
- 多重共线性 (Multicollinearity): 变量间高度相关, 导致参数估计方差极大。
- 过拟合 (Overfitting): 模型在训练集完美, 在测试集拉胯。
- 模型不可识别 (Unidentifiable): 当参数数量 $\geq$ 样本数量时, 无法求出唯一解。

7.1.2 2. 交互项的组合爆炸

交互项公式: 两个变量 X_1, X_2 的交互项是乘积项 $X_1 * X_2$ 。

系数解释 (β_3): 在模型 $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 (X_1 * X_2)$ 中, β_3 表示 X_1 对 Y 的影响如何随着 X_2 的变化而变化 (即斜率的斜率)。

主效应原则 (Hierarchy Principle) — 考试重点:

- **规则:** 如果模型中包含交互项 ($X_1 * X_2$), 必须同时包含其对应的所有主效应 (X_1 和 X_2)。
- **理由:** 即使主效应不显著, 也要留着, 否则模型的几何解释会由坐标轴变化而改变, 失去稳定性。

数量级估算: 包含所有交互项会导致 p 指数级爆炸 (2^p), 这是引入 PCA 的直接动机。

7.2 第二板块: PCA 的核心数学原理 (The Theory)

这里是概念题和计算逻辑题的重灾区。

7.2.1 1. PCA 的终极目标

核心目的: 找到一组新的预测变量 (Z), 它们是原始变量 (X) 的线性组合, 且能捕捉数据中最大的方差 (Maximum Variance)。

口诀：信息 = 方差。方差越大，包含的信息越多。

7.2.2 2. 主成分 (Principal Components, PCs) 的三大性质 (必背)

- 线性组合: $Z_i = \sum \phi_{ji} X_j$ 。
- 正交性 (Pairwise Orthogonal): 所有的 Z 之间互不相关 (Correlation = 0)。这解决了多重共线性问题。
- 有序性 (Ordered):
 - Z_1 (第一主成分): 捕捉最大方差。
 - Z_2 (第二主成分): 捕捉次大方差 (且垂直于 Z_1)。
 - ...
 - Z_p : 捕捉最小方差 (通常是噪音)。

7.2.3 3. 几何解释

- 投影 (Projection): PCA 本质上是將 n 个数据点投影到一个由前 m 个特征向量定义的新空间中。
- 旋转: 也可以理解为旋转坐标轴, 让新的坐标轴方向与数据分布最“长”的方向对齐。

7.3 第三板块: PCA 的算法实现与计算 (The Algorithm)

如果考代码填空或步骤排序, 就在这里出题。

7.3.1 1. 标准化 (Standardization) — 至关重要!

- 步骤: 在做 PCA 之前, 必须对每个 X 进行标准化。
- 要求: Mean = 0, Variance = 1。
- 原因: 如果不标准化, 量纲大 (数值大) 的变量 (如“工资”) 会主导方差, 掩盖量纲小 (如“年龄”) 的变量。PCA 会失效。

7.3.2 2. 线性代数实现步骤 (Step-by-Step)

1. 标准化: $X \rightarrow \text{scaled } X$ 。
2. 计算协方差矩阵/相关矩阵: $X^T X$ 。
3. 特征分解: 计算 $X^T X$ 的特征向量 (Eigenvectors) 和 特征值 (Eigenvalues)。
 - 特征向量 ($\mathbf{W}$): 定义了主成分的方向 (即权重 ϕ)。
 - 特征值: 定义了该主成分解释了多少方差。

4. 排序: 按特征值从大到小排列特征向量。
5. 映射 (Transform): 计算新得分 $Z = XW$ 。

7.4 第四板块: 主成分回归 (PCR) 与模型评估

这是 PCA 在实际回归任务中的应用, 也是考试中的“分析题”考点。

7.4.1 1. PCR 的流程

- 不要用所有 p 个 Z (那样没降维, 结果和 OLS 一样)。
- 截断: 只选择前 M 个主成分 $(Z_1, \dots, Z_M)$, 其中 $M < p$ 。
- 回归: 用这 M 个 Z 作为预测变量, 拟合线性回归模型预测 Y 。

7.4.2 2. 如何选择 M ? (超参数调优)

唯一标准答案: 交叉验证 (Cross-Validation)。我们选择能使 CV Error (Test MSE) 最小的那个 M 。

7.4.3 3. 解释方差比 (Explained Variance Ratio)

公式: $\frac{\text{Var}(Z_i)}{\sum \text{Total Variance}}$

用于评估每个主成分的重要性。通常画图 (Scree Plot) 来辅助决定 M 。

7.5 第五板块: 考试陷阱与优缺点 (The “Gotchas”)

这部分专门对付判断题和批判性思维题。

7.5.1 1. PCA/PCR 的缺点 (Cons) — 高频考点

无监督 (Unsupervised):

- PCA 计算全过程完全不看 Y 。
- 致命弱点: 方差最大的 Z_1 不一定与 Y 最相关! 有可能方差很小的 Z_{last} 才是对预测 Y 最有用的。因此 PCR 不一定能提高预测精度。

丧失解释性 (Interpretability):

- 原始回归系数 β_{age} 很容易解释 (年龄每增加1岁...)。
- PCR 的系数是针对 Z 的, 而 Z 是所有原始变量的混合体, 无法用人类语言直观解释。

7.5.2 2. PCA/PCR 的优点 (Pros)

- **解决多重共线性:** 因为 Z 之间正交。
- **降维:** 处理 $p > n$ 的情况。
- **可视化:** 将高维数据降到 2D/3D 进行绘图观察。

7.6 老师的最后叮嘱 (Exam Tips)

上考场前, 请闭上眼睛自问三个问题:

- **如果不标准化直接做 PCA 会发生什么?** (答: 方差大的变量会吃掉方差小的变量, 结果无效)。
- **PCA 是有监督还是无监督? 这对预测有什么潜在风险?** (答: 无监督。风险是主成分可能与 Y 无关)。
- **如果有交互项 $X_1 * X_2$, 能不能只把 X_1 删了?** (答: 不能, 必须遵守主效应原则)。

8 Lecture 8: Clustering (聚类分析) — 终极考前复习清单

8.1 第一部分：核心概念与“高维灾难” (Core Concepts & High-Dim Issues)

这部分通常出现在概念选择题或简答题的开头。

8.1.1 1. 聚类的定义 (Definition)

- **输入:** 一组点 (Set of points) + 距离度量 (Distance measure)。
- **目标:** 将点分组, 使得簇内 (Intra-cluster) 相似/距离近, 簇间 (Inter-cluster) 不相似/距离远。

8.1.2 2. 高维空间的挑战 (Curse of Dimensionality)

- **直觉误区:** 在二维或低维空间中, 聚类看起来很容易 (肉眼可见), 但在高维空间 ($10 \sim 10,000$ 维) 中完全不同。
- **距离失效:** 在高维空间中, 几乎任意两点之间的距离都差不多 (Almost all pairs of points are at about the same distance)。
- **考试考点:** 为什么在高维空间聚类很难? 答: 因为距离度量失去了区分度。

8.2 第二部分：数据表示与距离度量 (Representation & Distance)

必考点: 根据数据类型选择正确的距离公式。

8.2.1 三大对应关系 (必须要背)

- **集合 (Set of words)** $\rightarrow$ Jaccard Distance (杰卡德距离)。
- **空间点 (Point in space)** $\rightarrow$ Euclidean Distance (欧几里得距离)。
- **向量 (Vector)** $\rightarrow$ Cosine Distance (余弦距离)。

文档/CD的表示: 通常表示为高维空间中的向量或点 (例如: $x_i = 1$ 表示包含该词/购买该CD)。

8.3 第三部分：层次聚类 (Hierarchical Clustering)

这部分重点在于“如何合并”以及“非欧空间的特殊处理”。

8.3.1 1. 两种策略

- **凝聚式 (Agglomerative):** 自底向上 (Bottom-up), 从 N 个簇合并到 1 个。(本课重点)

- **分裂式 (Divisive):** 自顶向下 (Top-down), 从 1 个簇分裂到 N 个。

8.3.2 2. 核心步骤 (Key Operation)

重复计算所有簇之间的距离, 合并最近 (nearest) 的两个簇。

8.3.3 3. 簇的表示与距离计算 (关键区分)

情况 A: 欧几里得空间 (Euclidean Case)

- 使用 **质心 (Centroid)**: 簇内所有点的平均值 (Average)。
- **注意**: 质心通常是一个人造点 (Artificial point), 实际数据中可能不存在。
- **距离计算**: 计算两个质心之间的距离。

情况 B: 非欧几里得空间 (Non-Euclidean Case)

- 使用 **簇代表点 (Clustroid)**: 簇内现有的 (Existing) 某一个点, 它离簇内其他点“最近”。
- **Clustroid 的定义标准**:
 - 到其他点最大距离最小 (Smallest maximum distance)。
 - 到其他点平均距离最小 (Smallest average distance)。
 - 到其他点距离平方和最小 (Smallest sum of squares) (最常用公式: $\min_c \sum_{x \in C} d(x, c)^2$)。

8.3.4 4. 停止条件 (Termination Condition)

- **预设 K 值**: 当簇的数量达到预先设定的 k 时停止。
- **内聚性阈值 (Cohesion)**: 当下一次合并会导致“簇内聚性”过低 (即产生 Bad cluster) 时停止。
 - **内聚性的度量**: 直径 (Diameter)、半径 (Radius)、密度 (Density)。

8.4 第四部分: K-Means 算法 (K-Means Clustering)

全章节最重要的算法, 必须掌握流程和 K 值选择。

8.4.1 1. 算法流程 (The Lloyd's Algorithm)

1. **初始化**: 随机选 k 个点作为初始质心。
2. **分配 (Assignment)**: 遍历所有点, 将其分配给最近的质心。
3. **更新 (Update)**: 计算每个簇的新质心 (所有点的平均值)。
4. **循环**: 重复步骤 2 和 3, 直到收敛 (Convergence) (即点不再移动, 质心不再变)

化)。

8.4.2 2. 如何选择 K 值 (手肘法 — The Elbow Method)

- **画图:** 横轴 k , 纵轴“点到质心的平均距离”。
- **观察:** 随着 k 增大, 平均距离会下降。
- **结论:** 选择曲线下降变缓的拐点 (The point where average falls rapidly then changes little) 作为最佳 k 值。

8.4.3 3. 初始化策略 (Picking Initial Points)

- **坏策略:** 完全随机 (可能导致局部最优)。
- **好策略 1 (采样):** 对小样本先做层次聚类, 取结果的中心作为初始点。
- **好策略 2 (分散):** 选第一个点随机, 后续每个点都选离当前所有点最远的那个点 (Pick dispersed set of points)。

8.5 第五部分: 进阶算法 — 应对大数据与复杂形状

当 K-Means 失效时 (内存不够或形状不规则), 考这两个算法。

8.5.1 1. BFR 算法 (针对海量数据)

场景: 数据量太大, 无法一次装入内存 (Data doesn't fit in memory)。

核心机制:

- 分块读取数据 (Block processing)。
- 内存中只保留元数据 (Metadata), 而不是原始点。

局限性 (必考):

- **强假设:** 假设簇服从正态分布 (Gaussian distribution)。
- **形状限制:** 簇必须是轴对齐的椭圆 (Axis-aligned ellipses)。倾斜的椭圆或不规则形状无法处理。

8.5.2 2. CURE 算法 (针对不规则形状)

全称: Clustering Using REpresentatives。

核心机制:

- 不使用单一质心, 而是使用一组代表点 (Collection of representative points)。
- 这些点从簇中选取, 并向簇中心收缩 (Shrink towards center)。

优势:

- **形状无关:** 可以处理任何形状 (Any shape) 的簇 (如弯月形、环形、长条形)。
- 克服了 K-Means 和 BFR 只能处理球形/椭球形的缺点。

8.6 第六部分: 算法复杂度对比 (Complexity Checklist)

这是选择题或计算题中判断算法效率的关键。

- **朴素层次聚类:** $O(N^3)$ — 极慢, 不可用于大数据。适用场景: 小数据, 需要层级结构。
- **优化层次聚类:** $O(N^2 \log N)$ — 依然很慢 (使用优先队列)。适用场景: 中等数据。
- **K-Means (单轮):** $O(k \cdot N)$ — 每轮线性, 但轮数未知。适用场景: 工业界标准, 大数据。
- **BFR:** 线性 (单遍扫描) — 依赖正态分布假设。适用场景: 超大数据 (Disk-resident)。
- **CURE:** 适用场景: 复杂形状。

8.7 老师的最后叮嘱 (Final Tips)

- **区分 Centroid 和 Clustroid:** 考试时如果题目提到“文档聚类”或“非数值型数据”, 脑子里立刻反应出 Clustroid。
- **K-Means 迭代逻辑:** 如果出大题让你手推 K-Means, 记得是“先分配, 再算平均, 再分配”。
- **算法适用性:**
 - 形状奇怪? 选 CURE。
 - 内存不够? 选 BFR。
 - 不知道几类? 用 层次聚类 或 K-Means + 手肘法。

9 Lecture 9: 监督学习核心模型（回归与分类）终极复习清单

9.1 第一部分：核心概念框架 (Conceptual Framework)

这部分主要应对概念选择题（MCQ），考察对模型适用性的判断。

9.1.1 1. 监督学习 (Supervised Learning)

定义：利用有标签的训练集 (Labeled training set) 学习一个函数（模型），将特征映射到目标变量。

核心区分 (必考)：根据目标变量 (Target Variable) 的类型来选择模型。

- **回归 (Regression)：**目标变量是连续数值型 (Continuous/Numerical)。
 - 例子：股价预测、GDP、身高、寿命。
- **分类 (Classification)：**目标变量是离散类别型 (Discrete/Categorical)。
 - 例子：邮件是否垃圾 (0/1)、肿瘤良性/恶性、天气 (晴/雨)。

9.1.2 2. 数据预处理 (回归模型必做)

异常值 (Outliers)：回归对异常值非常敏感（会导致回归线倾斜），必须移除，但需确保不丢失有用信息。

归一化 (Normalization)：

- **原因：**特征尺度不同（如收入 vs 年龄）会导致权重 w 不可解释。
- **方法：**Z-score 标准化。
- **公式：**

$$z = \frac{x - \mu}{\sigma}$$

(减去均值，除以标准差)。

9.2 第二部分：线性回归模型 (Linear Regression)

这部分主要考察公式记忆、矩阵运算形式以及对参数含义的理解。

9.2.1 1. 模型定义

- **一维公式：** $f(x) = w_0 + w_1x$
- **多元公式：** $f(x) = w_0 + \sum_{j=1}^k w_jx_j$
- **优化目标：**最小化 SSE (误差平方和): $\sum (y_i - f(x_i))^2$

9.2.2 2. 核心计算公式 (背诵)

一维参数求解:

- 斜率:

$$w_1 = r_{xy} \frac{\sigma_y}{\sigma_x}$$

- 截距:

$$w_0 = \bar{y} - w_1 \bar{x}$$

多元参数求解 (矩阵闭式解 — 重点):

设 X 为特征矩阵 (加一列 1), y 为目标向量。

最优权重向量 w :

$$w = (X^T X)^{-1} X^T y$$

复杂模型扩展: 可以通过对输入取对数 ($\log x$) 或增加多项式项 (x^2) 来拟合非线性关系, 但本质仍是参数线性的。

9.2.3 3. 解释性

- **权重 w_j 含义:** 当其他特征保持不变时, x_j 每增加一个单位, y 的变化量。
- **显著性测试:** 零假设 $H_0: w_j = 0$ 。如果拒绝零假设, 说明该特征对预测结果有显著影响。

9.3 第三部分: 分类模型 — 决策树 (Decision Trees)

这部分是计算题的重灾区 (Gini/Entropy), 以及算法逻辑题的重点。

9.3.1 1. 树的结构与逻辑

组成:

- **根节点/内部节点:** 属性测试 (Test on an attribute)。
- **分支:** 测试结果 (Outcome)。
- **叶节点:** 类标签 (Class Label)。

构建策略:

- **贪心策略 (Greedy Strategy):** 每一步都选择当前看来最好的分裂, 不考虑全局最优。
- **复杂度:** 寻找全局最优树是 NP-hard 问题。

- **Hunt 算法:** 递归分裂, 直到节点“纯净”(只含一类) 或无属性可分 (用多数类标记)。

9.3.2 2. 节点分裂技术 (Splitting Techniques)

名义属性 (Nominal) (如颜色):

- **多路分裂:** 每个值一个分支。
- **二元分裂:** 需要尝试所有组合 (如 {红} vs {绿, 蓝}), 寻找最优。

有序属性 (Ordinal) (如 Size: S, M, L):

- **关键规则:** 分裂必须尊重顺序 (Respect Order)。
- **合法:** {S, M} vs {L}。
- **非法:** {S, L} vs {M} (不能跳跃)。

连续属性 (Continuous) (如 Income):

- **离散化:** 静态或动态分桶。
- **二元切分:** 寻找最佳切分点 v (如 $Income < 80k$)。

9.3.3 3. 纯度计算公式 (Calculation — 必须熟练手算)

我们希望节点越“纯”越好 (Impurity 越低越好)。

Gini Index (基尼系数 — CART算法):

- **公式:**

$$Gini(t) = 1 - \sum_i [p(i|t)]^2$$

- **计算口诀:** 1 减去 (各类别比例的平方和)。
- **范围:** 0 (最纯) $\sim$ 0.5 (二分类最乱)。

Entropy (熵 — ID3/C4.5算法):

- **公式:**

$$Entropy(t) = - \sum_i p(i|t) \log_2 p(i|t)$$

- **计算口诀:** 负的 (比例 $\times$ $\log$ 比例) 之和。
- **范围:** 0 (最纯) $\sim$ 1 (二分类最乱)。

Classification Error (分类误差):

- 公式:

$$Error(t) = 1 - \max(p(i|t))$$

- 计算口诀: 1 减去 (最大类别的比例)。

如何决定最佳分裂 (Best Split):

- 计算分裂前后的 增益 (Gain) = (父节点不纯度) - (子节点加权平均不纯度)。
- 选择增益最大 (或分裂后加权 Gini 最小) 的属性作为测试条件。

9.4 第四部分: 模型评估体系 (Evaluation)

这部分主要应对计算 Accuracy/Error rate 的题目。

9.4.1 1. 混淆矩阵 (Confusion Matrix)

	预测 Yes (1)	预测 No (0)
实际 Yes (1)	f_{11} (TP — 真阳)	f_{10} (FN — 假阴)
实际 No (0)	f_{01} (FP — 假阳)	f_{00} (TN — 真阴)

9.4.2 2. 核心指标公式

准确率 (Accuracy):

$$Accuracy = \frac{\text{预测对的数量}}{\text{总数量}} = \frac{f_{11} + f_{00}}{Total}$$

错误率 (Error Rate):

$$Error = \frac{\text{预测错的数量}}{\text{总数量}} = \frac{f_{10} + f_{01}}{Total}$$

9.5 考试避坑指南 (Exam Tips)

- **公式混淆:** 千万别把 Linear Regression 的闭式解 $(X^T X)^{-1} X^T y$ 记错了, 顺序不能乱。
- **Gini 计算:** 算 Gini 时记得是用 1 减去平方和。很多同学算完平方和就交卷了, 那是错的!
- **有序属性分裂:** 如果考题问“为什么 {Small, Large} vs {Medium} 这种分法不对?”, 答案一定是“违反了有序属性的顺序性 (Violates the order)”。
- **模型选择:** 题目给个场景让你选模型, 先看目标变量。是数字? 选回归。是类别 (Yes/No)? 选决策树/分类。

- **过拟合：**决策树如果不限深度，容易过拟合（每个叶子只有一个样本，纯度最高但泛化能力差）。这对应了 sklearn 参数中的 `max_depth`。

10 Lecture 10: Analysis Models & Evaluation — 终极考试特化 知识点清单

既然你已经刷了两套卷子，我们现在不谈虚的，直接进行考前“急救包”级别的知识点重构。我将整个 Lecture 10 的零散 PPT 页面打散，重新聚合为四大核心板块。这是你上考场前**必须死记硬背、理解透彻**的内容。如果不掌握这些，考试遇到计算题和概念辨析题会非常危险。

10.1 第一板块：核心分类算法 (The “Big Three” Algorithms)

这一板块涵盖了三种主要模型的原理、公式和考点。重点在于区分它们的决策边界和核心参数。

10.1.1 1. 最近邻分类器 (K-Nearest Neighbor, KNN)

核心机制：懒惰学习 (Lazy Learner)。不预先建立模型，预测时才计算。

必背公式 (欧几里得距离):

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

决策逻辑：

1. 计算未知点与所有训练点的距离。
2. 选出最近的 k 个点。
3. 多数投票 (Majority Vote) 决定类别。

关键考点 (k 值的选择):

- k 太小 (Small k , e.g., $k=1$):
 - 模型太复杂，决策边界破碎。
 - 对噪声极度敏感。
 - 导致 **过拟合 (Overfitting)** / 高方差 (High Variance)。
 - 概念: 1-NN 对应的边界图称为 Voronoi Diagram。
- k 太大 (Large k):
 - 模型太简单，决策边界过于平滑。
 - 容易包含其他类别的点。

- 导致 **欠拟合 (Underfitting)** / 高偏差 (High Bias)。

10.1.2 2. 支持向量机 (SVM)

核心目标: 寻找一个超平面, 使得分类间隔 (Margin) 最大化。

必背公式 (间隔计算):

$$Margin = \frac{2}{\|\vec{w}\|}$$

优化目标:

- 为了最大化间隔, 我们需要 **最小化** $\|\vec{w}\|$ (或 $\frac{\|\vec{w}\|^2}{2}$)。
- **硬间隔 (Hard Margin):** 要求所有样本必须分对, 且在间隔外: $y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1$ 。

软间隔 (Soft Margin) 与 参数 C:

- 引入: 当数据线性不可分或有噪声时, 引入松弛变量 ξ_i 。
- **新目标函数:** $Minimize: \frac{\|\vec{w}\|^2}{2} + C(\sum \xi_i)^k$ 。
- **参数 C 的影响 (高频考点):**
 - C 很大: 对错误零容忍 $\rightarrow$ 强制拟合所有点 $\rightarrow$ 容易过拟合。
 - C 很小: 容忍更多错误 $\rightarrow$ 间隔更宽 $\rightarrow$ 泛化能力可能更好, 但可能欠拟合。

核技巧 (Kernel Trick):

- 解决非线性问题。
- 思路: 将低维不可分数据映射到高维空间, 使其线性可分。

10.1.3 3. 逻辑回归 (Logistic Regression)

本质: 尽管叫回归, 其实是分类算法。输出的是概率。

核心函数 (Sigmoid):

$$P(C_+|x) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

将输出压缩在 $[0, 1]$ 之间。

线性关系 (必考):

虽然输出是非线性的, 但在对数几率 (Log-odds) 层面是线性的:

$$\log \frac{P(C_+|x)}{P(C_-|x)} = \vec{w} \cdot \vec{x} + b$$

求解方法:

- 使用 极大似然估计 (MLE)。
- 没有闭式解 (No closed form solution), 必须用梯度下降迭代求解。

10.2 第二板块: 模型理论与泛化 (Theory & Generalization)

这一板块解释了“为什么模型会表现不好”, 重点在于理解 U 型曲线和偏差-方差权衡。

10.2.1 1. 偏差-方差权衡 (Bias-Variance Tradeoff)

- **偏差 (Bias):** 模型在训练集上的拟合能力。
 - High Bias: 还没学会 (Underfitting)。模型太简单。
- **方差 (Variance):** 模型对训练数据变化的敏感度。
 - High Variance: 学傻了 (Overfitting)。模型太复杂, 死记硬背了噪声。
- **权衡:** 降低一个通常会升高另一个, 需寻找平衡点。

10.2.2 2. 过拟合 (Overfitting) 与 欠拟合 (Underfitting)

判断标准 (U型曲线图):

- **训练误差 (Training Error):** 随着模型变复杂, 永远在下降。
- **测试误差 (Test Error):** 随着模型变复杂, 先下降后上升 (U型)。

状态识别:

- **欠拟合:** 训练误差大, 测试误差大。
- **过拟合:** 训练误差极小, 但测试误差大 (Gap 很大)。

解决过拟合的大招:

- **增加训练数据量 (More Data):** 这是最有效的手段, 能缩小训练和测试误差的差距。
- **降低模型复杂度 (正则化、剪枝等)。**

10.3 第三板块: 核心评估指标 (Metric Calculation)

这一板块是 **计算题重灾区**。如果不背熟这些公式, 拿到混淆矩阵你也算不对。

10.3.1 1. 混淆矩阵 (Confusion Matrix)

	预测 Yes	预测 No
真实 Yes	TP (a)	FN (b)
真实 No	FP (c)	TN (d)

10.3.2 2. 必背计算公式

Accuracy (准确率):

$$Acc = \frac{TP + TN}{Total} = \frac{a + d}{a + b + c + d}$$

缺陷: 在数据不平衡 (Imbalanced Data) 时严重失真 (具有误导性)。

Precision (精确率/查准率): "预测为 Yes 的里面, 多少是真的?"

$$Precision = \frac{TP}{TP + FP} = \frac{a}{a + c}$$

Recall (召回率/查全率/TPR): "真实的 Yes 里面, 我找出了多少?"

$$Recall = \frac{TP}{TP + FN} = \frac{a}{a + b}$$

F1-Score: Precision 和 Recall 的调和平均。

$$F = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

10.3.3 3. ROC 曲线与 AUC

坐标轴 (千万别记反):

- **Y轴:** TPR (Recall / Sensitivity) → 越高越好。
- **X轴:** FPR ($1 - Specificity = \frac{FP}{TN+FP}$) → 越低越好。

关键点:

- **(0,1):** 左上角, 完美模型。
- **对角线:** 随机猜测 (Random Guessing)。

AUC (Area Under Curve):

- 衡量模型整体性能。AUC = 0.5 (瞎猜), AUC = 1.0 (完美)。
- **模型比较:** 如果两条 ROC 曲线交叉, 说明互有优劣 (一个在低误报时好, 一个

在高误报时好)。

10.4 第四板块：复杂场景与实战策略 (Advanced & Practical)

这一板块涉及”怎么切分数据”和”怎么算钱（成本）”。成本矩阵计算是极高概率出现的大题。

10.4.1 1. 成本矩阵计算 (Classification Cost)

核心逻辑：不同的错误代价不同 (例如：漏诊癌症比误诊更严重)。

计算套路：

1. 给定 Cost Matrix (例如 $C(FN) = 100, C(FP) = 1$)。
2. 给定 Confusion Matrix (TP, FN, FP, TN 的数量)。
3. 计算总成本:

$$Total\ Cost = (TP \times C_{TP}) + (FN \times C_{FN}) + (FP \times C_{FP}) + (TN \times C_{TN})$$

决策标准：选择 Total Cost 最低的模型，而不是 Accuracy 最高的模型。

10.4.2 2. 数据切分方法 (Validation Methods)

- **Holdout:** 简单的 2/3 训练，1/3 测试。
- **k-fold Cross Validation (k折交叉验证):** 数据分 k 份，轮流做测试集。保证每个样本都被测过。
- **Bootstrap:** 有放回采样。常数记忆: 约 63.2% 的数据用于训练，36.8% 用于测试。

10.4.3 3. 类别不平衡处理 (Class Imbalance)

问题：负样本 9990 个，正样本 10 个。模型全猜负，准确率 99.9%，但毫无意义。

对策：

- 不要看 Accuracy，看 Precision & Recall。
- **重采样 (Resampling):**
 - **上采样 (Oversampling):** 复制小类样本 (可能导致过拟合)。
 - **下采样 (Undersampling):** 减少大类样本。
- 使用 Cost Matrix 增加小类分错的惩罚权重。

10.5 老师的最后叮嘱 (Final Exam Tips)

- **公式不要张冠李戴:** 最大的坑是 Precision 分母是 $TP + FP$ (预测出的正类), Recall 分母是 $TP + FN$ (真实的正类)。考前最后一分钟再默写一遍。
- **K 与 C 的方向:**
 - KNN 的 k 越小 $\rightarrow$ 越复杂 $\rightarrow$ 越过拟合。
 - SVM 的 C 越大 $\rightarrow$ 越复杂 $\rightarrow$ 越过拟合。
 - 这两个参数对模型复杂度的影响方向是相反的记忆逻辑, 容易混淆。
- **计算题陷阱:** 遇到 Cost Matrix 的题目, 千万别凭直觉选准确率高的那个, 一定要按公式算出总 Cost 再比较。哪怕 Accuracy 只有 50%, 只要它在昂贵的错误上犯错少, 它就是好模型。

11 Lecture 11: Word Embeddings & Skip-gram Model — 考前终极复习

这份清单不再按照 PPT 页码排序，而是根据考点逻辑进行了聚类。请重点关注公式背诵、矩阵维度计算和算法流程。

11.1 一、模型架构与核心参数 (Architecture & Parameters)

这是选择题和填空题的**高频考点**，务必搞清维度。

11.1.1 1. 基本定义

- V (Vocabulary Size): 词汇表的大小（例如 10,000 个单词）。
- N (Embedding Dimension): 嵌入向量的维度（即 Hidden Layer 的大小，通常远小于 V ，例如 300）。
- Input Layer & Output Layer: 维度均为 V （对应 One-hot 向量）。

11.1.2 2. 两大核心权重矩阵 (必须死记维度)

W (Input Matrix / Context Matrix):

- 维度: $N \times V$ (或者 $V \times N$ ，取决于行/列向量定义，课件中 Input Layer 到 Hidden Layer 若为 $x^T W$ ，则 W 为 $V \times N$ ；若为 Wx ，则为 $N \times V$)。
- 关键理解: W 的每一列（或行）对应一个单词作为中心词 (Center Word) 时的向量表示。

W' (Output Matrix / Center Matrix):

- 维度: $V \times N$ (与 W 转置对应)。
- 关键理解: W' 的每一行（或列）对应一个单词作为背景词 (Context Word) 时的向量表示。

参数总量 (Parameter Count): 模型需要学习的参数总数为 $2 \times V \times N$ 。

11.1.3 3. 最终词向量的选择

训练结束后，我们可以选择：

- 只用 W （最常见）。
- 只用 W' 。
- 取平均值 $\frac{W+W'}{2}$ 。

11.2 二、Skip-gram 算法逻辑流程 (Algorithm Logic)

考试中可能要求描述过程或画简图。

11.2.1 1. 任务目标

- **Given:** 一个中心词 (Center Word) w_t 。
- **Predict:** 定义窗口半径 m 内的上下文单词 (Context Words) w_{t+j} 。

11.2.2 2. 前向传播五步走 (The 5 Steps)

1. **Input:** 生成中心词的 One-hot 向量 x^c 。
2. **Projection:** 通过 $v_c = Wx^c$ 获取中心词的嵌入向量 (即“查表”操作)。
3. **Dot Product:** 计算 $z = W'v_c$ 。这步得到一个 V 维的向量, 每个分量代表中心词与词库中某个词的“相似度得分”。
4. **Probability:** 使用 Softmax 将得分 z 转化为概率分布 $\hat{y} = \text{softmax}(z)$ 。
5. **Error:** 将预测概率 $\hat{y}$ 与真实背景词的 One-hot 向量进行比较 (计算 Loss)。

11.3 三、核心数学公式 (Key Formulas)

这是计算题和推导题的命脉, 符号必须准确。

11.3.1 1. Softmax 函数 (归一化概率)

给定中心词 c (center) 和某个特定的背景词 w (word/context):

$$p(w|c) = \frac{\exp(v_w'^T v_c)}{\sum_{k=1}^V \exp(v_k'^T v_c)}$$

考点解析:

- **分子:** 中心词向量 v_c 与目标背景词向量 v_w' 的点积 (Dot Product) 的指数。点积越大, 概率越大。
- **分母:** 归一化项。必须遍历整个词汇表 V (计算成本极高, 是优化的动力)。

11.3.2 2. 目标函数 (Objective Function)

最大化似然 (Likelihood):

$$J'(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} p(w_{t+j} | w_t; \theta)$$

最小化负对数似然 (Negative Log Likelihood — Loss Function): 这是最常用的形式

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log p(w_{t+j} | w_t; \theta)$$

解释: T 是语料库总词数, m 是窗口大小。我们需要最小化该 Loss 来更新 θ (即 W 和 W')。

11.4 四、词向量的特性与应用 (Properties & Applications)

通常出现在概念理解题或简答题中。

11.4.1 1. 语义相似度 (Similarity)

- 词向量在空间中的距离代表语义的相似程度。
- 位置靠近 = 语义相似 (例如 "politics" 和 "liberal")。

11.4.2 2. 线性类比关系 (Linear Analogies) [必考]

词向量空间保留了线性关系, 支持向量加减运算。

核心测试公式: $a : b :: c : ?$

$$\text{Target} \approx v_b - v_a + v_c$$

经典例子:

$$v_{King} - v_{Man} + v_{Woman} \approx v_{Queen}$$

含义: 向量差 $v_{King} - v_{Man}$ 捕获了"皇室/权力"的概念 (去除性别), 或者说 $v_{Woman} - v_{Man}$ 捕获了"性别转换"的向量方向。

11.4.3 3. 语法与语义捕捉

- Syntactic (语法): 复数关系 ($x_{apple} - x_{apples} \approx x_{car} - x_{cars}$)。
- Semantic (语义): 包含关系、性别关系等。

11.5 老师的考前叮嘱 (Final Check)

- 不要混淆 v 和 v' : 考试推导时, 一定要分清哪个是 Input Vector (来自 W), 哪个是 Output Vector (来自 W')。通常中心词用 v (Input), 背景词用 v' (Output)。
- 点积的物理意义: 如果问你为什么用 Dot Product, 答案是: 它是衡量两个向量方向一致性 (相似度) 最简单的方法。

- **计算量陷阱：** 如果考题问“为什么基础 Skip-gram 慢？”，答案在于 Softmax 的分母需要遍历整个词汇表 V ，计算量太大。

12 Lecture 12: 推荐系统核心理论与算法逻辑 — 终极复习

12.1 第一部分：核心商业概念与长尾理论 (The Long Tail)

这部分通常出现在选择题或简答题的背景理解中。

12.1.1 1. 从稀缺到富饶 (Scarcity to Abundance)

- **传统零售:** 货架空间 (Shelf space) 稀缺，只能展示热门商品。
- **Web 时代:** 分发成本近乎为零，商品极其丰富，导致用户选择困难。
- **结论:** 更多的选择需要更好的过滤器 (Filters) → 即推荐系统。

12.1.2 2. 长尾理论 (The Long Tail)

- **头部 (Head):** 少数极度热门的产品 (Hits)。
- **尾部 (Tail):** 数量巨大的小众产品 (Niche products)。
- **核心逻辑:** 当选择的可用性极高且分发成本可忽略时，需求会向“长尾”转移。
- **推荐系统的作用:** 挖掘长尾价值，向用户展示他们不知道自己需要的潜在兴趣 (Latent needs)。

12.2 第二部分：形式化模型与数据获取 (Formal Model)

理解数据的输入形式，是计算题的前提。

12.2.1 1. 效用矩阵 (Utility Matrix)

- **行 (Rows):** 用户 (Customers)。
- **列 (Cols):** 物品 (Items)。
- **值:** 评分 (Ratings, 如 1-5 星)。
- **性质:** 这是一个稀疏矩阵 (Sparse)，绝大多数位置是空的。

12.2.2 2. 数据获取方式

- **显式 (Explicit):** 用户主动打分 (准确但难获取)。
- **隐式 (Implicit):** 基于行为推断 (购买、点击、观看时长)。

12.3 第三部分：算法一 — 基于内容的推荐 (Content-Based)

核心逻辑: “如果你喜欢某物品，你会喜欢与它特征相似的物品。”

12.3.1 1. 物品画像 (Item Profile) 与 TF-IDF (必考计算)

这是本章最重要的特征提取算法公式。

概念: 将物品表示为特征向量。对于文本, 使用 TF-IDF 提取关键词。

公式 1: TF (词频):

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{kj}}$$

逻辑: 词 i 在文档 j 中出现的次数除以该文档中出现最多的词的次数。

目的: 归一化, 消除长文档带来的偏差。

公式 2: IDF (逆文档频率):

$$IDF_i = \log \frac{N}{n_i}$$

逻辑: N 是总文档数, n_i 是包含词 i 的文档数。

目的: 降低常用词 (如 "the", "is") 的权重, 提升稀有词的区分度。

公式 3: TF-IDF Score:

$$w_{ij} = TF_{ij} \times IDF_i$$

12.3.2 2. 用户画像 (User Profile)

- **计算逻辑:** 用户的画像 = 用户喜欢过的所有物品画像的加权平均值 (Weighted average)。
- **进阶处理:** 有时需要减去用户的平均评分进行归一化, 消除打分习惯差异。

12.3.3 3. 预测方法

计算用户画像向量与候选物品画像向量之间的 余弦相似度 (Cosine Similarity)。

12.4 第四部分: 算法二 — 协同过滤 (Collaborative Filtering)

核心逻辑: "利用群体的智慧 (Peer validation)"。

12.4.1 1. 两种模式

- **User-User CF:** 找和用户 x 口味相似的人 N , 看 N 喜欢什么。
- **Item-Item CF:** 找和物品 i 被同一拨人喜欢的相似物品 j 。
- **考试重点:** 工业界更倾向于 Item-Item, 因为物品比用户更稳定 (Static), 且易于分类。

12.4.2 2. 三大相似度计算公式 (Sim Metrics) — 死记硬背

考试中会给你一个矩阵，让你算两个向量的相似度。

A. 杰卡德相似度 (Jaccard Similarity):

$$Sim(A, B) = \frac{|r_A \cap r_B|}{|r_A \cup r_B|}$$

逻辑：交集除以并集。

缺陷：完全忽略评分的具体数值，只看是否评价过。

B. 余弦相似度 (Cosine Similarity):

$$Sim(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

(点积除以模长乘积)

缺陷：将缺失值视为 0（负面），无法区分“没看过”和“不喜欢”。

C. 中心化余弦 / 皮尔逊相关系数 (Centered Cosine / Pearson) — 最推荐

步骤 1 (归一化)：每个评分减去该行(用户)的平均分。

结果：正数=喜欢，负数=不喜欢，0=平均/缺失。

步骤 2 (计算)：对处理后的向量计算余弦相似度。

$$Sim(x, y) = \frac{\sum (r_{xi} - \bar{r}_x)(r_{yi} - \bar{r}_y)}{\sqrt{\sum (r_{xi} - \bar{r}_x)^2} \sqrt{\sum (r_{yi} - \bar{r}_y)^2}}$$

优势：解决了评分尺度不一（手松手紧）的问题，且完美处理缺失值。

12.5 第五部分：最终评分预测 (Rating Prediction)

算出相似度后，如何预测具体分数？这是大题的最后一步。

12.5.1 1. 核心公式 (加权平均)

$$r_{pred} = \frac{\sum_{j \in N} (Sim_{ij} \times r_j)}{\sum_{j \in N} Sim_{ij}}$$

- Sim_{ij} : 相似度（作为权重）。
- r_j : 邻居的实际评分。
- 逻辑：相似度越高的话语权越重。

12.5.2 2. Item-Item CF 计算流程 (考试高频题)

1. **Target:** 预测用户 U 对物品 I 的评分。
2. **Selection:** 找到用户 U 评价过的、且与物品 I 最相似的 k 个物品。
3. **Calculation:** 套用上述加权平均公式。

12.6 第六部分：评估体系 (Evaluation)

如何判断推荐系统好坏？区分回归问题和分类问题。

12.6.1 1. RMSE (均方根误差)

- **场景:** 预测具体评分 (回归问题)。
- **公式:** $\sqrt{\text{平均误差的平方}}$ 。
- **缺点:** 惩罚所有误差，甚至包括无关紧要的低分段误差。

12.6.2 2. Top-N 推荐评估 (分类问题)

Precision (查准率): $\frac{TP}{TP+FP}$

含义：系统推出来的里面，有多少是用户真正喜欢的？(用户视角的准确性)

Recall (查全率): $\frac{TP}{TP+FN}$

含义：用户真正喜欢的里面，有多少被系统推出来了？(系统覆盖的能力)

记忆口诀：Precision 分母是“预测出的正例”，Recall 分母是“真实的正例”。

12.7 第七部分：算法优缺点对比 (Pros & Cons) — 简答题必背

维度	基于内容 (Content-Based)	协同过滤 (CF)
原理	依靠物品本身特征 (Features)	依靠群体智慧 (User behaviors)
冷启动	无新物品冷启动 (新物品有特征即可推)	严重 (新物品/新用户无交互无法推)
多样性	过度专业化 (Overspecialization), 只推同类	有惊喜 (Serendipity), 能发现潜在兴趣
稀疏性	不受影响	受影响严重 (很难找到重合邻居)
应用	文本、电影等特征明显的领域	通用, 任何领域 (无需理解物品内容)
解释性	强 (因为你喜欢X, 所以推Y)	弱 (黑盒, 基于统计)

12.8 考前最后提醒 (Final Exam Tips)

- **计算题陷阱:** 在使用 User-based CF 或 Item-based CF 时，如果题目没有明确指定用哪种相似度，首选 Pearson (Centered Cosine)，因为它最严谨。

- 归一化方向:
 - **User-based:** 减去用户的平均分（消除打分习惯差异）。
 - **Item-based:** 减去物品的平均分（消除物品热门程度差异）。
- **TF-IDF:** 算 TF 时分母是该文档中最大词频，不是总词数。算 IDF 时记得取 Log。
- **混合推荐 (Hybrid):** 解决冷启动和稀疏性的终极方案是混合两种算法。